FLY | Collaborative AI Travel Agent — Complete Technical & Learning Handbook

From the professor's Google Colab notebook to a live Hugging Face PRO Gradio application

Project implementation guide

October 2026

FLY — Collaborative AI Travel Agent
Quick navigation

1. Executive overview
   1.1 The idea in one sentence
   1.2 What the project contains
   1.3 What was achieved, and what is still conditional
   1.4 Two different places where Fly runs

2. Essential concepts — explained simply
   2.1 Python notebook cell
   2.2 API
   2.3 LLM versus agent
   2.4 AWS, IAM and Bedrock
   2.5 Hugging Face Hub versus Spaces

3. The complete architecture
   3.1 Read the architecture from left to right
   3.2 Why the components are split
   3.3 The deployed directory structure
   3.4 Data flow for one traveler

4. The assignment and Human–AI Interaction goals
   4.1 What HAII Assignment 2 asks for
   4.2 Design requirements R1–R4
   4.3 Linking to Microsoft guidelines
   4.4 Evidence you should preserve

5. Professor notebook — cell by cell
   5.1 A necessary caveat about numbering
   5.2 Cell 5 — AWS configuration, concept and code
   5.3 Cell 6 — Model initialization
   5.4 Cell 8 — Custom agent tools
   5.5 Cell 9 — Baseline `CodeAgent`
   5.6 Cell 10 — Baseline Gradio
   5.7 Cells 11–12 — Inspecting evidence
   5.8 Cell 14 — Improved agent
   5.9 Cell 16 — Pink Fly interface
   5.10 Hugging Face publication cells

6. AWS Bedrock — authentication and model access, step by step
   6.1 What happens when you click Generate
   6.2 The AWS components involved
   6.3 Create or select an AWS IAM identity
   6.4 The least-privilege policy used by the project
   6.5 Check that the identity and region match
   6.6 Model identifiers: three related strings
   6.7 Bedrock model settings and the stop-sequence incident
   6.8 How AWS credentials reach Hugging Face
   6.9 AWS cost model

7. Smolagents, CodeAgent and tools — how the brain is coordinated
   7.1 Smolagents is an orchestration library
   7.2 Why a CodeAgent rather than a simple model call?
   7.3 What each tool contributes
   7.4 `max_steps` and practical latency
   7.5 Where the actual hosted agent is created

8. The improved local agent — Cell 14
   8.1 Design objective

8.2 Two conversational modes
8.3 Prompt engineering versus application logic
8.4 A typical local Cell 14 control flow
8.5 Why “one initial setup” does not mean one rigid answer

9. Pink Gradio interface — Cell 16
9.1 What Gradio does
9.2 Main UI regions
9.3 Custom trip duration
9.4 Typed interests
9.5 The normal chat Send button
9.6 Animated planning state
9.7 Gradio version compatibility
9.8 Live dashboard versus preview mode

10. Image enrichment and location links
10.1 Why the AI model does not directly generate the photographs
10.2 The image-enrichment pipeline
10.3 Why Wikimedia images can still fail
10.4 Why the image lookups run after the answer
10.5 Google Maps links are not bookings
10.6 Attribution and licensing

11. Conversation memory, session state and corrections
11.1 What Fly stores during a conversation
11.2 Memory is not the same as model training
11.3 What happens when the traveler corrects a preference
11.4 Why fresh agents are important
11.5 What is not stored permanently

12. Hugging Face hosting — from Static to live Gradio
12.1 How the publication problem started
12.2 Why Hugging Face PRO mattered
12.3 Why we created a new repository
12.4 Public, protected and private

13. Hugging Face backend — file-by-file walkthrough
13.1 `README.md` — Space metadata and operating instructions
13.2 `requirements.txt` — dependencies

13.3 `agent_backend.py` — the AI orchestration layer
13.4 `app.py` — frontend and request coordination
13.5 Why code exists both in notebook and Space

14. Deployment procedure — beginner walkthrough
14.1 Before touching the hosting setup
14.2 What to run in your existing Colab
14.3 Illustrative publisher API flow
14.4 Understand the previous YAML failure
14.5 Configure Space environment variables
14.6 First deployment check
14.7 Test in a safe sequence
14.8 How to apply the later image patch

15. Day-to-day operating procedures
15.1 Start and inspect the app
15.2 Rotate credentials
15.3 Patch code with minimal scope
15.4 Upgrade dependencies carefully
15.5 Operational health monitoring

16. Security, privacy, reliability and cost controls
16.1 Threats relevant to this architecture
16.2 Do not confuse Gradio password auth with full platform security
16.3 Handling sensitive user input
16.4 Source reliability hierarchy
16.5 Financial safeguards
16.6 Known technical limits of the inspected implementation

17. Quality assurance and professor's evaluation
17.1 Recommended test architecture
17.2 A/B test design
17.3 Quantitative metrics (with formulas)
17.4 Evidence table template
17.5 Functional acceptance checklist

18. Troubleshooting — error messages, likely cause and resolution
18.1 Troubleshooting decision tree

18.2 Error reference
18.3 Troubleshoot one layer at a time
18.4 How to write a useful bug report

19. Technical roadmap — beyond the current educational demo
19.1 Prioritized future improvements
19.2 Why validated structured output is a major improvement
19.3 Productionization is not just visual polish

20. Glossary and official references
20.1 Essential terminology
20.2 Official technical references
20.3 Project files inspected to produce this handbook

21. Practical learning exercises

Final project summary

FLY — Collaborative AI Travel Agent

Complete Technical and Learning Handbook | HAII Assignment 2
Edition: 1.0, 23 October 2026
Audience: Beginner to intermediate Python developers, Human–AI Interaction students, reviewers, and future project maintainers.
Purpose: Explain **what was built, why it was built, how every important component works, how the full system was deployed, and how to operate, test, secure, and improve it**.

> **Important distinction:** This document is grounded in the project conversation and the application source files available for review: the Colab replacement cells, `agent_backend.py`, `app.py`, the image-enhanced `app.py` variant, the Hugging Face deployment notebook, `requirements.txt`, and Space README. It does **not** assume that every unprovided professor notebook cell has a specific

> implementation, and it does not certify the live Space's current git revision or actual cloud availability. The live Space should be checked independently against the published commit.

Quick navigation

1. Executive overview and outcome
2. Beginner's explanation of the technology
3. End-to-end system architecture
4. Original assignment and Human–AI Interaction objectives
5. Professor notebook: cell-by-cell understanding
6. AWS IAM, authentication, Bedrock and DeepSeek V3.2
7. Smolagents: how the agent reasons and uses tools
8. The improved agent: Cell 14
9. The pink Gradio dashboard: Cell 16
10. Deep dive into image enrichment and links
11. How conversational memory really works
12. Hugging Face: Static versus live Gradio hosting
13. Hosted backend source: file-by-file
14. Deployment workflow, start to finish
15. Operating and maintaining the deployed app
16. Security, privacy, limits and costs
17. Testing, experimental evidence, acceptance tests
18. Troubleshooting knowledge base
19. Future improvements and technical roadmap
20. Glossary and official references
21. End-to-end practical exercises

---

1. Executive overview

1.1 The idea in one sentence

Fly is a collaborative AI travel concierge that collects a traveler's preferences in a pink web dashboard, uses DeepSeek V3.2 on AWS Bedrock to create a tailored itinerary, adds optional real place photographs and links, and supports follow-up questions without automatically rebuilding the full itinerary.

Imagine a human travel agent. You first explain your destination, budget, trip length and interests. The agent produces a proposed itinerary. You may then ask, "Could we spend less on Day 2?" A good concierge answers that question in context instead of starting the original questionnaire again. Fly aims to reproduce that interaction pattern.

1.2 What the project contains

Layer	Concrete technology	Main responsibility
User experience	**Gradio 6.5.1** + custom CSS	Pink dashboard, travel form, chat history, buttons, loading feedback and photo cards
AI agent orchestrator	**Smolagents 1.26.0**, `CodeAgent`	Interprets tasks, decides whether to use web tools, obtains a response
AI model adapter	**LiteLLM** through `LiteLLMModel`	Translates model requests into a provider-compatible call
Remote model	**DeepSeek V3.2** through AWS Bedrock	Generates itinerary suggestions and conversational answers
Research tools	`WebSearchTool`, `VisitWebpageTool`	Optional search and page reading when current information matters
Media enrichment	Wikipedia / Wikimedia via `requests`	Optional destination images, attraction images and article links
Maps links	Google Maps search URL	Provides directions/search entry points, **not** a confirmed reservation
Hosting	**Hugging Face Gradio Space**	Runs Python app after Colab is closed
Credentials	Hugging Face Space Secrets; Colab Secrets during development	Prevents hard-coded AWS and Hugging Face tokens
State	Gradio `State`	Keeps a travel brief, saved original plan and recent dialogue for an active session

1.3 What was achieved, and what is still conditional

Reported as working by the project owner: The original DeepSeek V3.2 Bedrock call in Colab, the original baseline Gradio assistant, the improved conversational Fly agent locally, and the hosted Hugging Face Gradio app after AWS secrets were configured. The hosted image feature was initially missing; a patched `app.py` was prepared to improve Wikimedia lookups. Do not treat that patch as confirmed deployed until its uploaded revision and image test are checked.

Not guaranteed: every travel recommendation is correct, every user query succeeds, every place has a matching Wikimedia photo, every photo loads in every browser, exact prices, opening times, transport times or booking availability. This is an educational AI planning assistant, not a travel-booking engine.

1.4 Two different places where Fly runs

Development mode: Google Colab runs notebook Cells 5, 6, 8, 14 and 16, including the Gradio server. A `share=True` URL exists only while that Colab app is running.

Deployed mode: Hugging Face runs `app.py` and `agent_backend.py` from its own repository. It does not load Python variables from your Colab notebook. It receives its AWS secrets through Hugging Face Settings.

---

2. Essential concepts — explained simply

2.1 Python notebook cell

A **cell** is a block of Python executed inside Google Colab. Cells share a Python environment during one Colab runtime: a variable defined in Cell 6 can be referenced in Cell 14. That variable disappears when the runtime restarts. This explains why "I did not change Cell 14" does **not** mean you can skip rerunning it after a restart.

2.2 API

An **application programming interface (API)** allows software to request a service from other software. Here, Fly sends a model request to Bedrock; AWS processes it and returns a result. The API is not the visual website: the API is the interface between programs.

2.3 LLM versus agent

- **LLM / model:** generates text from input context. DeepSeek V3.2 is this language-model component.
- **Agent:** wraps the model in a control loop and provides tools. The Smolagents `CodeAgent` decides what action to take, calls tools when necessary and produces the final answer.
- **UI:** makes the system usable by a traveler. Gradio is the user interface, not the LLM.
- **Hosting:** keeps the UI and Python backend available remotely. Hugging Face Spaces supplies that runtime, not the DeepSeek model's inference.

Think of them as **brain (model) → coordinator (agent) → reception desk (UI) → building (hosting)**. This is a teaching analogy, not a statement that any AI system has a biological brain.

2.4 AWS, IAM and Bedrock

AWS is the cloud provider; **IAM** manages who is authorized; **Bedrock** provides model API endpoints; **DeepSeek V3.2** is the selected remote model. A Hugging Face account does **not** automatically authorize calls to AWS.

2.5 Hugging Face Hub versus Spaces

The **Hub repository** stores source files and model/agent artifacts. A **Space** can also run an application. Its `sdk` determines whether it hosts static HTML, Gradio/Python or a Docker application. Simply seeing the source files online does not prove that a Python chatbot is running.

---

3. The complete architecture

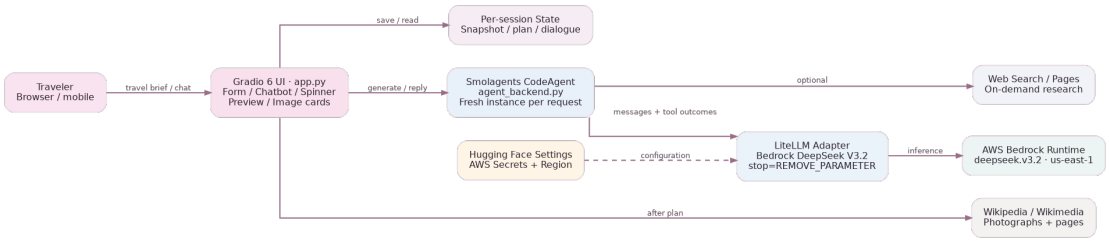


Figure 1 — End-to-end Fly architecture

3.1 Read the architecture from left to right

1. A traveler enters a trip brief or chat message in the **browser**.
2. **Gradio** receives the values and calls a Python callback.
3. The callback validates input and loads the active visitor's **session state**.
4. A fresh **Smolagents** `CodeAgent` is created for the request.
5. The CodeAgent calls **LiteLLM**, which forwards the prompt to **AWS Bedrock / DeepSeek V3.2**.
6. The agent can optionally access **web-search and webpage tools**, within its configured step limit.
7. The final itinerary or answer returns to Gradio.

8. Gradio displays the plan, **then separately attempts optional Wikimedia images and Maps links**.
9. For a follow-up question, Gradio sends the saved trip brief, original plan and a bounded recent conversation context.

3.2 Why the components are split

Keeping the front end and back end separate improves maintainability: changes to pink styling should not require rewriting AWS model setup, while changing a model ID should not require rewriting every textbox. It also makes faults easier to locate: a working preview with a failed AI call points toward model/backend configuration, not basic Gradio rendering.

3.3 The deployed directory structure

```
Hugging Face Space: haii-fly-travel-live/
├── README.md           # Space YAML metadata and operating instructions
├── requirements.txt     # Python dependencies installed at build time
├── agent_backend.py     # Bedrock model + CodeAgent factory and instructions
└── app.py               # Gradio dashboard, state, callbacks, images and launch
```

The Space uses these files at the **repository root**. The professor's notebook and your `HF_TOKEN` are **not** required at runtime and should not be copied into the repository merely to make this app execute.

3.4 Data flow for one traveler

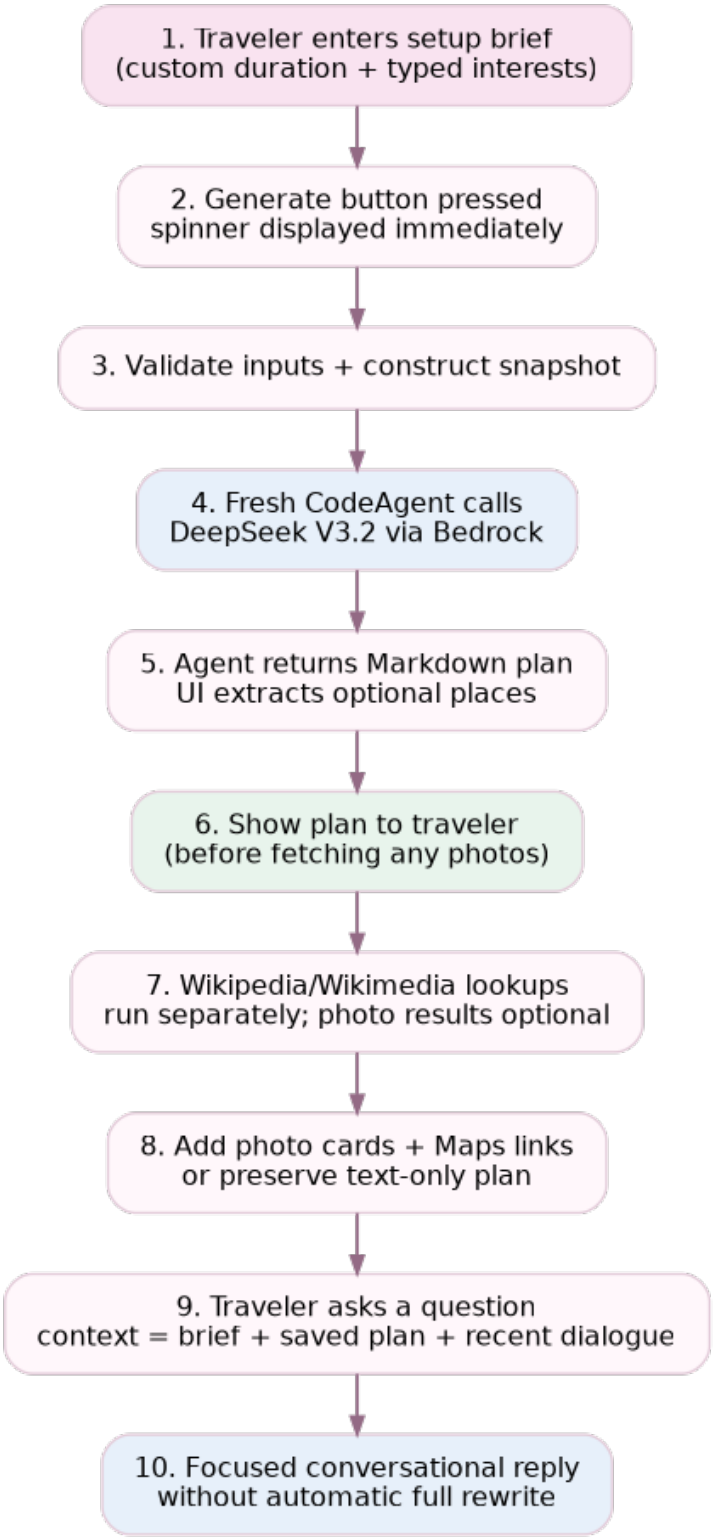


Figure 2 — Generate, enrich, then converse

The crucial user-experience optimization is **two-stage output**: show the textual itinerary as soon as the agent finishes, then add optional images. A slow Wikipedia response should not force the user to wait before seeing an otherwise complete plan.

---

4. The assignment and Human–AI Interaction goals

4.1 What HAII Assignment 2 asks for

The assignment is not merely "build a chatbot." It asks for a comparison between a baseline interaction and a deliberately improved one: identify observed limitations, connect them to human–AI guidelines, update requirements, implement changes and provide empirical before/after examples.

The initial baseline followed broad travel-planning instructions but the user reported several weaknesses, including **asking too many questions together** and **failing to explain recommendations**. These are observations from the project work, not independent formal measurements.

4.2 Design requirements R1–R4

ID	Human-centered requirement	How Fly addresses it	What would demonstrate it
R1	Collect essential personalization information once at the start	Dashboard takes destination, duration, budget, interests, typed interests and notes	No redundant multi-question interrogation after Generate
R2	Explain relevant recommendations	Agent instructions request a short "Why it fits" rationale	Count of major recommended stops with preference-based explanations
R3	Show a useful, visually clear itinerary	Day-by-day Markdown, pink cards, images when verifiably available, links	Usable day headings, place names and working reference links

ID	Human-centered requirement	How Fly addresses it	What would demonstrate it
R4	Support control, correction and conversation	Normal chat box and Send button; previous brief, plan and recent dialogue supplied as context	A question about Day 2 produces a focused answer without unsolicited full-plan rewrite

4.3 Linking to Microsoft guidelines

Guideline	Official meaning	Application to Fly
G1	Make clear what the system can do	Form hints, examples, and visible travel-planning scope
G2	Make clear how well it can do it	Disclose uncertainty, source limitations and estimated versus verified facts
G4	Show contextually relevant information	Prefer traveler-specific recommendations over generic travel encyclopedias
G8	Support efficient dismissal	Keep optional details nonintrusive; allow user not to follow suggestions
G9	Support efficient correction	Accept explicit preferences and follow-up changes in chat
G10	Scope services when in doubt	Use reasonable labeled assumptions rather than overstepping uncertain facts
G11	Make clear why the system did what it did	Explain the preference-to-recommendation connection
G12	Remember recent interactions	Recent chat is included in follow-up prompts
G14	Update and adapt cautiously	New explicit corrections override uncertain old inferences

Nuance: Adding a photo or shortening an answer does not **by itself** prove a Microsoft guideline was violated or satisfied. Formal evaluation requires observing a concrete usability problem and testing an appropriate change.

4.4 Evidence you should preserve

Save the original baseline user query, the **actual** baseline transcript, the updated-agent query (identical wording when possible), the improved transcript, and objective observations: number of questions asked, presence/absence of reasons, number of major suggestions, whether correc-

tions are respected, and whether useful links/images load. Do not manufacture baseline defects or use offline preview text as evidence of real model performance.

---

5. Professor notebook — cell by cell

5.1 A necessary caveat about numbering

The conversation establishes the roles of **Cells 5, 6, 8, 9, 10, 11, 12, 14, 16, 19 and 21**. Other cell numbers may contain installation code, assignment instructions, placeholders or publication guidance, but their exact full text was not supplied in the accessible source. The table intentionally avoids inventing their contents. In your actual Colab notebook, headings may shift when you insert extra cells.

Cell / stage	Role	Depends on	Produces
0–4 (professor setup, variable)	Imports, packages, introductory text, notebook setup	Colab	Installed libraries and instructions
5	AWS credentials and region configuration	Colab Secrets, IAM user	Working AWS session / environment
6	DeepSeek V3.2 initialization and small model test	5, Smolagents, LiteLLM	`assignment_model` and compatibility alias `deepseek_r1_model`
8	Tool definitions	6, Smolagents / LiteLLM	Profiling tools, common-ground tool and `UserInputTool`
9	Professor baseline agent	6, 8	`travel_assistant_base`
10	Baseline Gradio interface	9	Temporary baseline chat URL
11	Replay previous baseline steps	9 / an agent run	Diagnostic replay
12	Read agent memory/step messages	9 / agent run	Internal execution observations
14	Improved collaborative agent	6, 8	`collaborative_travel_assistant`
16	Improved Fly Gradio dashboard	14	Travel form, chat, itinerary and temporary URL
19 (professor / deployment)	User-specific Hugging Face repository or upload code	HF token + exported files	Published repository / Space
21 (professor write-up)	Assignment write-up and evaluation	Experiment evidence	Final conclusions

Cell / stage	Role	Depends on	Produces
Added **HF deployment Cells 17–19**	New publishing helper cells (names may duplicate original numbering)	HF token + ZIP	Live `haii-fly-travel-live` Space

Why two different “Cell 19” references exist

Your professor’s original notebook and the extra deployment cells both use cell-number labels. In Colab, execution order is what matters, not the number printed in a comment. **Use function names and outputs to identify the correct cell.**

5.2 Cell 5 — AWS configuration, concept and code

The working Colab environment retrieved **AWS Access Key ID** and **AWS Secret Access Key** from Google Colab Secrets, then used `boto3.Session` configured for `us-east-1`. A private STS identity test proved that the key pair was valid. The original debugging history included a signature mismatch, later corrected by ensuring the key and secret belonged to the same credential pair and a fresh session was used.

Safe illustrative pattern (not a replacement for your already working Cell 5):

```
from google.colab import userdata
import boto3

aws_session = boto3.Session(
    aws_access_key_id=userdata.get("AWS_ACCESS_KEY_ID"),
    aws_secret_access_key=userdata.get("AWS_SECRET_ACCESS_KEY"),
    region_name="us-east-1",
)
# Avoid printing credentials.
print("AWS session configured")
```

What a secret is: a value accessible to the notebook runtime but not written into the notebook source code. It is not the same as a normal Python string pasted into a publicly shareable file.

5.3 Cell 6 — Model initialization

The model ID is `deepseek.v3.2` on Bedrock. LiteLLM's provider-prefixed form is `bedrock/deepseek.v3.2`. The crucial fix is to **omit** unsupported `stopSequences` using Smolagents' sentinel `REMOVE_PARAMETER`:

```python
import litellm
from smolagents import LiteLLMModel, REMOVE_PARAMETER

litellm.drop_params = True
assignment_model = LiteLLMModel(
    model_id="bedrock/deepseek.v3.2",
    stop=REMOVE_PARAMETER,
    max_tokens=1500,
)
deepseek_r1_model = assignment_model  # Legacy notebook compatibility only
```

Why this matters: The original interface could generate a simple answer, but a `CodeAgent` request included `stopSequences`; Bedrock returned a BadRequest. Setting `stop=[]` would not necessarily solve this, because the **parameter must be absent**, not an empty value.

Notebook trap: The alias `deepseek_r1_model` is a **variable name** from an earlier version of the notebook; it does **not** mean the model is DeepSeek-R1. Inspect `assignment_model.model_id` to identify the real model.

5.4 Cell 8 — Custom agent tools

Three components were defined:

1. `infer_task_commonground_from_conversation` — accepts conversation history and summarizes established goals or decisions. It calls the same model through `litellm.completion` when useful.
2. `infer_user_profile_from_conversation` — summarizes explicit and tentative user preferences; the design should avoid sensitive or unwarranted inferences.
3. `UserInputTool` — formats a context + question for the professor's interactive baseline UI. It works with the baseline's pause/response convention.

An `@tool`-decorated function declares arguments and return values so Smolagents can call it. Tools extend the model's abilities; they do **not** guarantee that the model will invoke them on every request.

Important implementation difference: The currently inspected Hugging Face `agent_backend.py` **does not register the two notebook summarization tools**; it registers only `WebSearchTool` and `VisitWebpageTool`. The hosted app supplies personalization directly through the structured travel brief and per-session dialogue. This is similar in user intent but **not an identical tool graph** to the professor notebook. Note the distinction if documenting implementation parity for the assignment.

5.5 Cell 9 — Baseline `CodeAgent`

Cell 9 created `travel_assistant_base` with the configured model and professor's tools. Its instructions encourage proactive research, personalization and sense-making before asking the traveler to choose. The agent also uses `max_steps=10` in the version shared during the project.

Why keep it: A before/after comparison needs an unmodified baseline. Replacing the original agent's instructions with the redesigned ones would undermine the experiment.

5.6 Cell 10 — Baseline Gradio

This cell wraps `travel_assistant_base` using Smolagents `GradioUI` and launches it. The student encountered a `gradio.components.Component` attribute error caused by a local package compatibility/inconsistency issue. That is distinct from a Bedrock authentication error. A clean dependency installation is preferable to long-term monkey-patching.

`debug=True` **vs** `debug=False`**:** A Gradio cell can remain executing while it runs a server. In Colab this may block the user's attempt to run later cells. To permit later cells to execute, stop the session or use a non-blocking launch with appropriate `prevent_thread_lock=True` behavior. Do not confuse stopping the Gradio server with losing every prior notebook variable; restarting the **runtime** is what clears the environment.

5.7 Cells 11–12 — Inspecting evidence

`travel_assistant_base.replay()` and inspection of `travel_assistant_base.memory.steps` help show **which tool calls and intermediate actions actually occurred**. These are useful for diagnosing unnecessary searches and linking observed behavior to HAI principles. They are **not the same as a traveler-visible final transcript**; preserve both.

5.8 Cell 14 — Improved agent

Cell 14 creates `collaborative_travel_assistant`, reuses the DeepSeek V3.2 model and introduces the redesigned interaction policy. Earlier versions included custom profiling tools; later conversational editions emphasized concise plans and normal follow-up answers.

The final local conversational version explicitly separates **Mode A: itinerary building** and **Mode B: focused conversation**, with lower step limits and restrained search. This policy is explained in Chapter 8.

5.9 Cell 16 — Pink Fly interface

Cell 16 implements the new dashboard, input controls, live status, chatbot and optional place links and images. It changed several times because of Gradio version changes, callback behavior and UX feedback. The final relevant features include **Custom duration**, **Other interests**, a normal **Send** textbox, **Preview**, **Design my journey**, **Start over**, and a normal follow-up chat that does not automatically rebuild the whole plan.

5.10 Hugging Face publication cells

The project first attempted to create a Gradio Space without an eligible plan and received **HTTP 402 Payment Required**. A free Static Space was used to publish files, but could not run Python. After Hugging Face PRO was enabled, a new Gradio Space was created using the standalone application package, and AWS credentials were configured as Space Secrets.

A subsequent upload failed at `/api/validate-yaml` because the README `short_description` exceeded the Hub's character limit; the YAML field was shortened and re-uploaded. These are deployment and configuration issues, not failures of the agent's travel logic.

---

6. AWS Bedrock — authentication and model access, step by step

6.1 What happens when you click Generate

A generated itinerary is not computed by Hugging Face. The Hugging Face Python process constructs a prompt, Smolagents asks LiteLLM to invoke Bedrock, AWS checks the supplied credentials and IAM policy, and DeepSeek V3.2 returns a response. The response then travels back along the same route to the browser.

6.2 The AWS components involved

AWS concept	Beginner-friendly meaning	Fly usage
AWS account	The billing and administration boundary	Owns the Bedrock permissions and charges
Region	Physical/geographic endpoint selection	Working setup used `us-east-1`
IAM identity	An AWS user/role whose privileges are controlled	Signs requests; should have least privilege
Access key ID	Public identifier of one credential pair	Set as secret `AWS_ACCESS_KEY_ID`
Secret access key	Private signing secret of that pair	Set as secret `AWS_SECRET_ACCESS_KEY`
Session token	Additional token for **temporary** STS credentials	`AWS_SESSION_TOKEN`, when relevant
STS	AWS service used to verify assumed/session identity	Useful for initial authentication checks
Bedrock	Hosted API for foundation models	DeepSeek V3.2 is called from Python
CloudWatch / billing	Monitoring services	Useful for usage, failures and costs

6.3 Create or select an AWS IAM identity

In the AWS Console, open **IAM → Users** (or your organization's approved identity mechanism). For a teaching demo, avoid the AWS root account. Prefer a dedicated, low-privilege identity used only by this application. If your organization prohibits long-lived IAM keys, use its approved short-lived credentials or federated mechanism instead. Such a mechanism requires a separate secure refresh/deployment setup beyond this current implementation.

The exact access-key creation flow may vary with AWS UI and policy. When a secret key is first generated, copy it into a suitable private secret manager; the value cannot normally be displayed again later.

6.4 The least-privilege policy used by the project

The project implemented permissions for model invocation on the regional foundation-model ARN, along these lines:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "InvokeDeepSeekV32Only",
      "Effect": "Allow",
      "Action": [
        "bedrock:InvokeModel",
        "bedrock:InvokeModelWithResponseStream"
      ],
      "Resource": "arn:aws:bedrock:us-east-1::foundation-model/deepseek.v3.2"
    }
  ]
}
```

Read this JSON as a rule: "Allow the named model-invocation API actions, against the named DeepSeek foundation model, in this region." The actual effective permission also depends on account policies, service control policies, possible explicit denies, and the Bedrock API path used. Remove streaming permission if it is unnecessary; do not broaden to `bedrock:*` or `Resource: *` unless separately justified.

The project initially encountered `AccessDeniedException` before the correct policy was attached to the IAM identity. Later, the model test succeeded.

6.5 Check that the identity and region match

A diagnostic workflow separates three questions:

1. **Authentication:** Are these keys valid and correctly paired? A private `sts.get_caller_identity()` test can help answer this.
2. **Authorization:** Does the identity have permission to invoke `deepseek.v3.2` in the chosen region?
3. **Compatibility:** Is the request structured in a way that DeepSeek V3.2 accepts?

A `SignatureDoesNotMatch` points to credentials/signing/session setup, while `AccessDeniedException` normally points to permissions or access conditions. A `stopSequences` BadRequest is neither: it is a model request compatibility problem.

6.6 Model identifiers: three related strings

String	Where used	Meaning
`deepseek.v3.2`	AWS Bedrock model ID	The model identifier AWS recognizes
`bedrock/deepseek.v3.2`	LiteLLM/Smolagents configuration	Adds the provider prefix so LiteLLM routes the call
`arn:aws:bedrock:us-east-1::foundation-model/deepseek.v3.2`	IAM policy resource	Identifies the model resource for authorization

A common beginner mistake is mixing these contexts, for example placing an ARN where a LiteLLM model ID is expected.

6.7 Bedrock model settings and the stop-sequence incident

`CodeAgent` can emit Python actions across several steps. Frameworks often send “stop” tokens to delimit generated content. In the user’s model route, Bedrock rejected the `stopSequences` field:

```
litellm.BadRequestError
This model doesn't support the stopSequences field.
Remove stopSequences and try again.
```

This was fixed with:

```python
from smolagents import LiteLLMModel, REMOVE_PARAMETER

assignment_model = LiteLLMModel(
    model_id="bedrock/deepseek.v3.2",
    stop=REMOVE_PARAMETER,
    max_tokens=1500,
)
```

`REMOVE_PARAMETER` is a **special framework sentinel**, not the literal string `"REMOVE_PARAMETER"`. It instructs Smolagents to omit the unsupported request field. An empty list can still result in an unwanted field reaching the provider. This was central to making the agent, not just an isolated model call, work.

6.8 How AWS credentials reach Hugging Face

Hugging Face Space **Settings → Secrets** stores the values in the application runtime environment. The hosted app uses the normal AWS SDK credential discovery chain through LiteLLM / Boto3; it does not need to import `google.colab.userdata`.

Name	Scope	Required?	Example / explanation
`AWS_ACCESS_KEY_ID`	HF Secret	Yes for present key-based setup	Actual AWS access key ID, never printed
`AWS_SECRET_ACCESS_KEY`	HF Secret	Yes	Its matching private signing key
`AWS_REGION`	HF Variable	Yes in practice	`us-east-1` in the working configuration
`AWS_SESSION_TOKEN`	HF Secret	Only for temporary credentials	Temporary, expires
`FLY_APP_PASSWORD`	HF Secret	Strongly recommended	App-level password
`FLY_APP_USER`	HF Secret / variable	Optional	Defaults to `traveler` in the deployed app
`HF_TOKEN`	**Colab Secret**, for publisher	For programmatic upload only	Does not need to be copied into the running Space

Security note: “Secret” means private platform configuration, **not** a promise that a malicious Python process with runtime access cannot read it. Least privilege, dependency review and application isolation still matter.

6.9 AWS cost model

Hugging Face PRO and AWS Bedrock are different bills. PRO provides eligibility for the chosen hosting environment; the language-model requests are charged by AWS according to the model/pricing/region and usage.

A useful rough model is:

[*i* (*P*{in} + P_{out})]

where `T_in` and `T_out` are input and output tokens for each request and `P_in`, `P_out` are the applicable **current** per-million-token prices. This is a generic estimator; it does not include other tool/network/hosting charges or any tier-specific pricing adjustments. Check the provider’s current pricing and budgets before public access.

Important design consequence: A 4-step maximum can constrain agent activity but does not guarantee low cost. Large prompts, repeated chat context and tool retries can still create substantial traffic.

---

7. Smolagents, CodeAgent and tools — how the brain is coordinated

7.1 Smolagents is an orchestration library

Smolagents is a Python framework that connects a model to a tool-based action loop. It receives a task, asks the model what to do, executes permitted tool or code actions and repeats until a final answer or a stopping condition.

The approximate reasoning-control loop is:

```
Traveler task
   ↓
```

```
Agent instructions + current context
   ↓
DeepSeek generates an action or final answer
   ↓
If action: CodeAgent executes the authorized tool/code step
   ↓
Tool result is added to context
   ↓
Model continues or returns final_answer
```

The model may reason internally, but the user-facing **"Why this fits"** is a concise justification of recommendations, not a demand to expose hidden chain-of-thought.

7.2 Why a CodeAgent rather than a simple model call?

A simple call can answer from general knowledge but cannot autonomously retrieve a web page. A `CodeAgent` can invoke `WebSearchTool` to find a source and `VisitWebpageTool` to read it. The trade-off is **more execution complexity, more latency and higher security requirements**, because model-generated Python code is executed by the framework.

This is why a live public CodeAgent should not be treated as fully isolated merely because it has an application password. For a production-grade deployment, use a suitably hardened execution environment and restrict external tools, network access and credentials.

7.3 What each tool contributes

Tool	Input	Typical output	Common failure
`WebSearchTool(max_results=2)`	Search query	Ranked search snippets / links	Rate limit, missing dependency, stale results
`VisitWebpageTool()`	Page URL	Extracted page text	Site blocks fetch, redirects, extraction problems
Baseline `UserInputTool`	Context + question	A formatted question	Not suited to a standalone synchronous travel form
Baseline profile summarizer	Conversation text	Tentative preference summary	Incorrect inference / model error
Baseline common-ground summarizer	Conversation text	Decisions, shared task state	Missing context, extra model latency

Not every request needs research. A question such as “Could you make Day 2 more relaxed?” can normally be answered from the saved plan. A question about **tomorrow’s opening hours** should ideally consult an authoritative current source rather than rely on model memory.

7.4 `max_steps` and practical latency

The original baseline used up to **10 agent steps**. The deployed `make_collaborative_travel_agent()` uses `max_steps=4`, with `verbosity_level=0`, to reduce potentially unnecessary tool/model loops. The local notebook’s earlier versions used different limits, including 5 or 8. These values refer to different historical code versions and are not contradictory.

What fewer steps do: cap the number of agent loop steps and generally reduce opportunities for repeated searching. **What they do not do:** guarantee a specific response time, guaranteed correctness, or a faster AWS model response per call.

7.5 Where the actual hosted agent is created

The hosted `agent_backend.py` defines a factory:

```
def make_collaborative_travel_agent():
    return CodeAgent(
        model=_FLY_MODEL,
        tools=[WebSearchTool(max_results=2), VisitWebpageTool()],
        name="collaborative_travel_assistant",
        instructions=FLY_INSTRUCTIONS,
        verbosity_level=0,
        max_steps=4,
    )
```

Factory means “a function that creates a new object.” A fresh agent for each UI request is safer than reusing one globally mutable agent across independent visitors. The hosted app stores context in its own `gr.State` object and explicitly feeds the relevant context to the new agent.

---

8. The improved local agent — Cell 14

8.1 Design objective

Cell 14 moves from a broad, occasionally overwhelming planning assistant toward a **collaborative** travel agent with a clear interaction contract: collect enough preferences once, explain recommendations, show a manageable plan and handle corrections without restarting the entire experience.

8.2 Two conversational modes

Mode A — Build itinerary. The UI has already obtained a structured trip brief. The agent generates a day-by-day itinerary immediately and makes optional assumptions explicit. The deployed instructions request approximately 1–3 specific places/activities per day with a short preference-based rationale.

Mode B — Conversational reply. The user asks a specific question after the plan exists. The agent should answer that question first, usually briefly, and not output a new complete itinerary unless explicitly requested.

Example:

- Initial request: “Four days in Kyoto, temples and peaceful gardens, relaxed pace.”
- Mode A: returns four concise day sections plus explanations.
- Follow-up: “Is there a quieter alternative for Day 2?”
- Mode B: proposes one or two alternatives for Day 2, explains why, and leaves unrelated days alone.

8.3 Prompt engineering versus application logic

A prompt tells the model how to behave; **application code enforces data collection, validation, state and UI events**. For instance:

- Prompt: “Do not ask a second questionnaire.”

- UI code: already collects destination and interests in the initial form.
- Prompt: “Respect corrections.”
- UI code: keeps recent dialogue and provides it to the next model request.
- Prompt: “Provide 3 attractions as a structured list.”
- UI code: parses a small optional fenced block to look up photos.

A prompt is **not a perfect enforcement mechanism**. Rigorous systems should validate the agent output and provide predictable fallbacks rather than assume the model always complies.

8.4 A typical local Cell 14 control flow

```
from smolagents import CodeAgent, WebSearchTool, VisitWebpageTool

collaborative_travel_assistant = CodeAgent(
    model=assignment_model,
    tools=[WebSearchTool(max_results=2), VisitWebpageTool()],
    name="collaborative_travel_assistant",
    instructions="...Mode A, Mode B, rationale and correction rules...",
    verbosity_level=1,
    max_steps=5,
)
```

This is a **short conceptual excerpt**, not the full current replacement cell. To reproduce the exact local behavior, use the saved code from `HAII_Cell_14_Fly_Conversational_Agent.py` rather than trying to execute this pseudocode.

8.5 Why “one initial setup” does not mean one rigid answer

The user originally wanted only one friendly personalization stage. The final pink form offers both convenient controls and free-text extensions. The required cognitive load is reduced because optional details need not be filled and the agent can make labeled assumptions. The traveler then has a chat input for natural follow-up questions.

---

9. Pink Gradio interface — Cell 16

9.1 What Gradio does

Gradio creates browser-facing components in Python. A `gr.Textbox` displays a text field, a `gr.Dropdown` provides choices, a `gr.Chatbot` renders messages, and a `gr.Button` invokes a Python callback through `.click()`.

A tiny conceptual Gradio example:

```
import gradio as gr

with gr.Blocks() as demo:
    question = gr.Textbox(label="Ask a question")
    answer = gr.Textbox(label="Response")
    send = gr.Button("Send")
    send.click(lambda text: "You asked: " + text, inputs=question, outputs=answer)

demo.launch()  # simplified learning example
```

Fly uses the same pattern at much greater scale: several inputs, progress updates, a chat session and a backend model call.

9.2 Main UI regions

Region	Visible elements	UX objective
Pink header	Fly branding / headline	Establish clear identity and friendly tone
Discovery brief	Destination, duration, trip dates, group, interests, budget, notes	One initial personalization stage
Travel summary card	Destination, duration, optional cover image and preference tags	Show the current journey at a glance
Status card	Idle, planning, ready, error	Show activity and prevent apparently dead buttons
Concierge chat	History of assistant and user messages	Natural two-way conversation
Message composer	Text field + **Send** button	Low-friction follow-up interaction
Places gallery	Photos + Wikipedia + Maps links	Make concrete recommendations easier to explore
Example buttons	Tokyo, Kyoto, Paris	Help a beginner discover usable starting prompts
Reset control	Start over	Clearly create a new trip session

9.3 Custom trip duration

The traveler may choose predefined durations or **Custom**, then type values such as `8 days` or `2 weeks`. In the hosted app, `_resolve_duration()` validates this using a regular expression and converts weeks to days:

```
# Simplified idea; actual implementation includes validation and bounds.
# "2 weeks" → "14 days (2 weeks)"
```

The current hosted version limits the accepted duration to **1–30 days**. Some forms may also accept “night(s)” as typed input; a careful product should distinguish days from nights because they are not semantically identical for lodging. In the inspected code, a numeric “night” value is currently treated as an equivalent count for itinerary days; this is a minor limitation to improve.

9.4 Typed interests

A checkbox group provides common interests, and `Other interests` accepts free-text interests such as *independent bookstores, quiet tea houses, architecture, jazz, street photography*. `_trip_snapshot()` merges both into the traveler snapshot passed to the agent.

This is important because forcing all users into predefined categories is a personalization limitation. A proper UI should allow people to express unique interests without replacing common easy-to-select options.

9.5 The normal chat Send button

The earlier “Refine Your Journey” block was replaced by a normal conversational text box. The user now asks direct questions: “How can I travel without a car?”, “What is cheaper near this attraction?”, “Could you make Day 3 quieter?” The **Send** action is connected to `chat_send()`; it does not call the full itinerary generator by default.

This behavior is a central distinction between a **form that generates documents** and a **collaborative agent**.

9.6 Animated planning state

When generation begins, the callback **yields** an intermediate result. The UI immediately displays a spinner and a temporary “Fly is designing your journey” message. Later yields replace that placeholder with the completed plan and eventually the optional photo appendix.

Why a generator helps: A normal Python callback that returns once at the end can look frozen for the duration of model inference and web research. A generator-based callback (`yield`) gives the browser observable state transitions without waiting for the final result.

This is **incremental UI feedback**, not necessarily token-by-token streaming of the model’s raw output.

9.7 Gradio version compatibility

The project encountered:

```
TypeError: Chatbot.__init__() got an unexpected keyword argument 'type'
```

In Gradio 6.x, omit the old `type="messages"` argument when constructing `gr.Chatbot`; the modern message format is handled natively. The project also encountered warnings that the `theme` and `css` options had moved from `gr.Blocks(...)` to `.launch(...)`.

Correct Gradio 6.x pattern:

```
with gr.Blocks(title="Fly") as fly_demo:
    chatbot = gr.Chatbot()

fly_demo.launch(theme=gr.themes.Soft(), css=FLY_CSS)
```

In the hosted app, `server_name="0.0.0.0"`, `server_port=7860`, `share=False` are used because Hugging Face itself supplies the public hosting URL. In Colab, `share=True` may be appropriate for a temporary demonstration; the two environments are intentionally different.

9.8 Live dashboard versus preview mode

- **Preview (no AWS):** Returns a clearly marked sample itinerary. It may still contact Wikimedia for images but does **not** call the Bedrock model. This tests interface construction and rendering independently.
- **Design my journey:** Calls the real AWS-backed agent, then optionally enriches the response.
- **Send:** Calls the agent again, but requests a focused contextual answer.
- **Start over:** Resets the current visible conversation and per-session state.

Do not record Preview output as evidence that the live DeepSeek model produced that response.

---

10. Image enrichment and location links

10.1 Why the AI model does not directly generate the photographs

DeepSeek V3.2 in this setup is a **text model**. Fly obtains images from a separate external content source: Wikipedia/Wikimedia. This separation makes it possible to show identifiable real destination photos rather than invented links or images that do not correspond to the place.

10.2 The image-enrichment pipeline

1. The agent writes a normal day-by-day itinerary.
2. The optional output contains up to three lines inside a fenced `places` block, e.g. `Senso-ji`, `Shinjuku Gyoen`.
3. `app.py` removes that machine-readable block from the traveler-facing text.
4. If the block is missing, `_separate_places()` can attempt a fallback extraction from bold place names.

5. `_place_visuals()` looks up the destination and selected attractions through `_wiki_media()`.
6. `_wiki_media()` attempts a Wikipedia summary and search/PageImages fallbacks, checks that place names plausibly match, and accepts photos only from trusted Wikimedia domains.
7. The UI displays image cards and article/Maps links when available. If the external lookup fails, the travel plan remains accessible.

10.3 Why Wikimedia images can still fail

Images may be absent because no article photo exists, the title is spelled differently, a network request times out, Wikipedia changes its API behavior, the image URL is blocked, the matched result is not trusted, or the browser declines to load a remote URL. **No working implementation can guarantee photographs for every destination.**

The first hosted deployment was reported to lack photographs compared with Colab. A later patch expanded lookup methods, allowed normalized title matching (for example, accented spellings) and added a destination-photo fallback. The patched file was provided separately as `Fly_HF_Images_Enabled_app.py` and should replace only `app.py` once deployed and tested.

10.4 Why the image lookups run after the answer

Image retrieval is **optional presentation enrichment**, not a dependency for the itinerary. In the inspected patched app:

```
First yield  → spinner / "working"
Model returns → yield text itinerary
Media lookup  → Wikipedia/Wikimedia requests
Final yield  → itinerary + linked photos, if available
```

Some photo lookups run concurrently in a `ThreadPoolExecutor(max_workers=...)`, with request timeouts. This can improve photo-loading latency but does not eliminate network variability.

10.5 Google Maps links are not bookings

The app constructs Maps **search** URLs with safely URL-encoded place names. The links help locate a destination; they do not confirm a venue is open, establish its actual availability, or make a reservation. This distinction must be communicated clearly for a trustworthy travel planner.

10.6 Attribution and licensing

Link back to the source encyclopedia page where possible and respect the specific Wikimedia media file's license and attribution requirements. A photograph's presence on Wikimedia is not permission to ignore its license. A mature production interface should offer explicit author/license attribution linked to the image's file page, not only the article page.

---

11. Conversation memory, session state and corrections

11.1 What Fly stores during a conversation

The deployed `app.py` uses a state structure equivalent to:

```
{
    "built": False,
    "snapshot": {},
    "plan": "",
    "dialogue": []
}
```

Here is what the keys mean:

Field	Meaning	Example
`built`	Whether an initial itinerary has succeeded	`True`
`snapshot`	Confirmed initial travel brief	Destination, days, interests and budget
`plan`	Original full itinerary generated by Mode A	Day 1 … Day 4

Field	Meaning	Example
`dialogue`	Recent user/assistant turns	“Can Day 2 be cheaper?” and answer

11.2 Memory is not the same as model training

The system **does not retrain DeepSeek V3.2** on the user’s preferences. Instead, it **adds relevant previous context to later prompts**. This is contextual conversational memory, not permanent machine-learning model personalization.

In a current follow-up request, `chat_send()` includes the original brief, approximately the first `6,800` characters of the saved plan, and a bounded number of dialogue entries. The code keeps only the last 12 role messages in state, and the prompt includes a smaller recent slice. This keeps prompts from growing without limit, but older decisions can fall out of context.

11.3 What happens when the traveler corrects a preference

Example:

> “Actually, I prefer a central hotel even if the area is busy.”

The current application appends that correction to recent dialogue and includes it in subsequent requests. The agent’s instructions explicitly tell it to prioritize the latest **explicit** correction. It should not confuse a tentative inference (“you may like rural hotels”) with a confirmed statement (“I prefer central hotels”).

Important implementation limit: This conversational correction is not automatically written back into every field of the structured `snapshot`. The original `plan` may also remain unchanged. Therefore, the current system gives the user a **focused conversational answer**, not necessarily a durable re-generated document. If a correction must persist indefinitely or

update the dashboard controls, the future implementation should include a structured preference-update routine and possibly an explicit “Apply this change to itinerary” action.

11.4 Why fresh agents are important

A single globally shared `CodeAgent` might accidentally carry previous runtime memory into another visitor’s chat if reused carelessly. The hosted app instead calls `make_collaborative_travel_agent()` for each request and passes an explicit **per-session** state.

The inspected app also uses a process-level `threading.Lock()` around agent execution. This serializes requests to a shared model adapter and helps reduce overlapping mutation, but **it also limits concurrency** and can make one user wait for another. That is a functional capacity limitation for a busy public app.

11.5 What is not stored permanently

This educational implementation does **not** include a database, user accounts with stored travel itineraries, persistent encrypted trip records, long-term preference profiles, synchronization across devices, or a background scheduler. Refreshes, new sessions or a server restart can lose state. Persistent memory would require a separate storage design, privacy policy and data-retention controls.

---

12. Hugging Face hosting — from Static to live Gradio

12.1 How the publication problem started

The professor's illustrative publication example created a Space with `space_sdk="static"` and attempted to `push_to_hub()`. A **Static Space** is suitable for HTML/JS files, not for executing a Python Gradio server and AWS-backed Smolagents. That is why the source repository could be visible while the chatbot did not actually run there.

The project then explored a free Static Space as a temporary code-hosting alternative. This was acceptable for **source publication**, but insufficient for the user's goal of letting classmates chat with Fly directly on Hugging Face.

12.2 Why Hugging Face PRO mattered

Creating a Gradio/Docker Space on the CPU Basic tier required an eligible paid account in this deployment period. The earlier account returned:

```
402 Payment Required
Static Spaces are free; compute-based Gradio/Docker Space
creation on CPU Basic requires an eligible subscription.
```

After the user enabled PRO, a live Gradio Space could be created. **CPU Basic is adequate for the current architecture because the language model runs remotely on AWS** rather than executing model weights on the Space's CPU or GPU. Compute and subscription rules can change; consult current Hugging Face documentation for the definitive requirements.

12.3 Why we created a new repository

The old source-only Space used:

```
melmir/haii_collaborative_travel_assistant
```

The standalone live Gradio application was designed for:

```
melmir/haii-fly-travel-live
```

These are **two different repositories**. The first is source-oriented / static; the second hosts the running Python chatbot. This avoids changing the SDK and content of a previously published educational artifact unexpectedly.

These URLs identify the intended project namespace; verify the exact authenticated account and repository in Colab's `HfApi.whoami()` and the Space's Files tab.

12.4 Public, protected and private

Visibility	Source visibility	Running app visibility	What it means for classmates
Private	Owner/collaborators only	Owner/collaborators only	Classmates cannot visit unless granted access
Protected (eligible paid plans)	Private source	App exposed via embed URL / custom domain	Suitable for sharing a live app while keeping source private, **not a substitute for login**
Public	Public	Public	Any visitor may use it; risk of AWS inference charges

The app's separate `FLY_APP_PASSWORD` authentication can restrict casual use. However, passwords, per-user quotas, hosting visibility and AWS budget limits serve different purposes. For a classroom demo, keep it private while testing, then choose a controlled sharing method approved by the assignment and your security requirements.

---

13. Hugging Face backend — file-by-file walkthrough

13.1 `README.md` — Space metadata and operating instructions

The first lines between `---` delimiters are **YAML metadata** read by Hugging Face:

```
***
title: Fly - Collaborative Travel Concierge
emoji: ✈️
colorFrom: pink
colorTo: pink
sdk: gradio
sdk_version: 6.5.1
app_file: app.py
python_version: "3.11"
suggested_hardware: cpu-basic
short_description: AI travel planner powered by DeepSeek V3.2
***
```

This is an explanatory corrected example, not necessarily identical to the current remote README. In the original package the `short_description` exceeded the Hub's allowed length and produced a validation error. The actual deployment repair shortened the field before uploading.

Why YAML matters: The Hub uses it to decide which runtime to start, which app file to launch, and which SDK version to install. A malformed YAML header can block a deployment before Python starts.

13.2 `requirements.txt` — dependencies

The inspected deployment package contains:

```
gradio==6.5.1
smolagents[litellm]==1.26.0
litellm>=1.75.0,<2
boto3>=1.35,<2
requests>=2.31,<3
beautifulsoup4>=4.12,<5
markdownify>=0.13,<2
ddgs>=9,<11
```

Why pin versions? Interface APIs change. Your Gradio 6 migration errors showed the risk: code written for Gradio 5 may pass unsupported constructor parameters in Gradio 6. Pinning exact Gradio/Smolagents versions makes rebuilding more reproducible.

Technical nuance: Range constraints such as `litellm>=1.75.0,<2` still permit future library releases, so this file is **partially pinned**, not a full immutable lockfile. A more reproducible production release should record a tested dependency lock with resolved versions and hashes.

13.3 `agent_backend.py` — the AI orchestration layer

Core responsibilities:

- Read `AWS_REGION` from environment variables.
- Initialize the shared LiteLLM-backed DeepSeek model.
- Include `stop=REMOVE_PARAMETER` to avoid the unsupported Bedrock field.
- Define the detailed `FLY_INSTRUCTIONS` with itinerary and conversation modes.
- Create a **fresh CodeAgent per request** with search and webpage tools.
- Limit the agent loop to 4 steps in the inspected deployment.

Importantly, the backend does **not** have a hardcoded AWS key and does **not** import Colab Secrets. Its AWS identity comes from the Space environment.

Detailed responsibility split

Function / object	Responsibility
`AWS_REGION`	Determine AWS model endpoint region
`assignment_model` / `_FLY_MODEL`	Configure the DeepSeek V3.2 LiteLLM adapter
`FLY_INSTRUCTIONS`	Behavioral contract for generation and conversation
`make_collaborative_travel_agent()`	Instantiate Smolagents CodeAgent with tools
`collaborative_travel_assistant = None`	Placeholder diagnostic name; live app uses the factory, not a singleton agent

13.4 `app.py` — frontend and request coordination

The inspected image-enhanced app contains these functions (names matter when you debug):

Function	What it does
`_new_state()`	Creates a fresh per-session state dictionary
`_txt()`	Normalizes user-provided values and defaults
`_status()`	Produces safe visual status messages, spinner/error/ready
`_maps_link()`	Builds encoded Google Maps search URLs
`_wiki_media()`	Retrieves plausible Wikipedia articles and Wikimedia photos
`_cover()`	Draws the destination cover card
`_resolve_duration()`	Validates and normalizes custom duration
`_trip_snapshot()`	Validates form values and builds structured preference record
`_brief()`	Converts snapshot to agent-readable text
`_run_fly()`	Creates a fresh agent, calls it and checks output
`_safe_error()`	Shows a user-safe error without exposing raw provider details
`_separate_places()`	Hides optional `places` metadata from user-facing plan
`_place_visuals()`	Builds photo cards and informational links
`build_trip()`	Main **Generate** callback, with multiple UI updates
`chat_send()`	Main conversational **Send** callback
`example_tokyo/kyoto/paris()`	Populate form example values
`preview()`	Non-AWS UI preview with optional Wikimedia media
`reset()`	Clears travel inputs, chat and session state
`show_custom()`	Shows custom duration box when selected

13.5 Why code exists both in notebook and Space

The notebook implementation is an **experimental, interactive development environment**. The Space implementation is a **self-contained deployment**. The hosting process starts from `app.py`, which imports its own backend. That separation is necessary because the Space cannot reference Colab's `assignment_model`, `userdata.get(...)` or notebook cell execution history.

This is also why changing Cell 16 in Colab **does not automatically update** the live Space. You must upload an updated `app.py` (and `agent_backend.py` if backend behavior changes) to the Space repository.

---

14. Deployment procedure — beginner walkthrough

14.1 Before touching the hosting setup

Have ready: an active Hugging Face account with the necessary hosting entitlement, an HF fine-grained token with write access to your personal repository, a working AWS IAM credential pair with Bedrock model permission, an approved AWS region, and the standalone deployment ZIP.

Do not share your AWS or Hugging Face secrets in a notebook output, screenshot, repository, or classroom group chat. If any historical credentials were exposed, rotate them before production deployment.

14.2 What to run in your existing Colab

You **do not need to rerun Cell 14 and Cell 16 to upload a standalone ZIP**. If Cell 16 still hosts a Gradio server, stop that running cell so the notebook can continue. Do not unnecessarily restart the runtime.

The deployment workflow added three cells at the bottom:

Cell 17 — HF authentication. Read `HF_TOKEN` from Colab Secrets, create `HfApi(token=...)`, call `whoami()` and build the repository ID.

Cell 18 — ZIP upload. Use `google.colab.files.upload()`, select `Fly_Travel_Agent_HF_Pro_Gradio_Space.zip`, and verify its root contains `app.py`, `agent_backend.py`, `requirements.txt`, `README.md`.

Cell 19 — Gradio Space publication. Create a new private Space using `create_repo(..., repo_type="space", space_sdk="gradio")`, upload the four required files, and verify the repository file list.

14.3 Illustrative publisher API flow

```
from google.colab import userdata
from huggingface_hub import HfApi

hf_api = HfApi(token=userdata.get("HF_TOKEN"))
username = hf_api.whoami()["name"]
repo_id = f"{username}/haii-fly-travel-live"

hf_api.create_repo(
    repo_id=repo_id,
    repo_type="space",
    space_sdk="gradio",
    private=True,
    exist_ok=True,
)

# The real publication cell then extracts/validates the ZIP and
# calls hf_api.upload_folder(...), committing app.py etc.
```

Do **not** use the excerpt alone as a full publisher; it only illustrates the creation step. The actual project cells also validate and upload files.

14.4 Understand the previous YAML failure

The first uploaded README had a `short_description` longer than the API's 60-character limit, so the Hub returned:

```
HTTP 400 Bad Request
Invalid metadata in README.md
"short_description" length must be less than or equal to 60 characters
```

A revised Cell 19 rewrites that one YAML line before upload. This teaches an important distinction: the Hub rejected the **repository metadata**, not the Python application code. Use the error's exact validation message to fix the responsible layer.

14.5 Configure Space environment variables

In Hugging Face, open the new **Gradio Space → Settings → Variables and secrets**. Set:

```
Secrets:
  AWS_ACCESS_KEY_ID       = [private value]
  AWS_SECRET_ACCESS_KEY   = [private value]
  FLY_APP_PASSWORD        = [strong private password]
  FLY_APP_USER            = [optional username]
  AWS_SESSION_TOKEN       = [only if temporary AWS credentials]
```

```
Variables:
  AWS_REGION              = us-east-1
```

Only the owner should enter the actual secret values. The region should match the AWS Bedrock region where the account and model permissions work. Changes to Space settings may restart the application.

14.6 First deployment check

Check **Space → Files** for the four expected files. Check the Space status:

- **Building:** Dependencies are being installed or the new commit is being deployed.
- **Running:** The application server has started; this does **not** prove AWS inference or images work.
- **Build Error:** Typically dependency, packaging, YAML or installation failure.
- **Runtime Error:** Python crashed or failed after installation.

Open **Space → Logs** for build/runtime errors. Avoid sharing full logs if they include tokens, usernames or account identifiers.

14.7 Test in a safe sequence

1. Log in to the app if Gradio password access is enabled.
2. Use **Preview (no AWS)**: verifies UI and optional media functions without paid model requests.
3. Click an example such as **Kyoto** to populate form values.
4. Click **Design my journey**: tests end-to-end Bedrock generation.
5. Ask “How can I get around Kyoto without renting a car?” in the normal chat input: tests Mode B.
6. Use **Start over**: tests reset and state isolation.
7. Test with a second browser/private session: ensure the other visitor’s itinerary does not appear.

14.8 How to apply the later image patch

The separate image patch is a replacement for **Space** `app.py`, not for your Colab Cell 14, your Space `agent_backend.py` or your AWS secrets. Upload the patch bytes to `path_in_repo="app.py"`, which creates a new repository commit and rebuilds the Space. Test Preview and a real trip again afterward.

Release-management recommendation: Save a snapshot of the previously working commit before patching. If the new deployment fails, revert to the prior commit and inspect the difference. Do not repeatedly overwrite several files while troubleshooting one image problem.

---

15. Day-to-day operating procedures

15.1 Start and inspect the app

Open the live Space address, check its App view and inspect Logs if the app is building or failing. Because the running server is independent from Google Colab, you do not have to keep your notebook or local computer switched on for normal hosted operation.

15.2 Rotate credentials

When rotating an AWS access key:

1. Create a replacement key for the **same authorized IAM identity** (or update access architecture appropriately).
2. Update `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` together in Space Secrets.
3. Add or update session token if using temporary credentials.
4. Allow the Space to restart and verify one controlled model request.
5. Disable the old key and confirm that traffic uses the new one.

Never commit keys into `requirements.txt`, `README.md`, `app.py`, log messages or public source files.

15.3 Patch code with minimal scope

Change you want	Edit this file	Why
Pink colors, sizing or layout	`app.py` (`FLY_CSS`, Gradio layout)	UI concern
New travel-form input	`app.py`	Form inputs and snapshot validation
More concise answers	`agent_backend.py` (`FLY_INSTRUCTIONS`, possibly limits)	AI behavior
Change model ID	`agent_backend.py` and matching IAM settings	Backend/model concern
New Python package	`requirements.txt`	Build dependencies
App title or SDK version	`README.md` YAML	Space metadata
New photo source / attribution	`app.py` image helpers	Media enrichment
AWS key rotation	Space **Secrets**, not code	Security and operations

15.4 Upgrade dependencies carefully

Do not upgrade every package automatically. Create a test branch or duplicate Space, adjust one dependency at a time, rebuild, run the full acceptance checklist and only then promote. The Gradio 5-to-6 constructor change illustrates why framework upgrades can break a previously stable interface.

15.5 Operational health monitoring

A mature operator should track model request count, error rate by layer, typical latency, successful itinerary count, failed image lookups, session-reset incidents and estimated Bedrock spending. Currently the educational app has visible status and server logs, but **does not include a full observability stack with metrics and alerts**.

---

16. Security, privacy, reliability and cost controls

16.1 Threats relevant to this architecture

Risk	Why it matters	Existing or recommended control
Exposed AWS credentials	Unauthorized Bedrock charges and access	HF Secrets, key rotation, no secrets in repo
Excessively broad IAM privileges	Compromise could reach other AWS resources	Scope model invocation to required resources
Public anonymous abuse	Anyone could generate paid model requests	Space visibility, login, quotas, AWS budgets
Prompt injection through web pages	Untrusted pages may contain malicious instructions	Treat tool content as untrusted, limit tools, harden execution
Model-generated Python execution	CodeAgent runs actions, not just text	Isolated sandbox, low-privilege runtime, limited network/filesystem
Cross-user memory leakage	One visitor may see another visitor's data	Fresh agents, Gradio per-session state, isolation tests
Personal travel data disclosure	Inputs can include itinerary, family/health details	Minimize input, retention policy, restricted access
Hallucinated travel facts	Incorrect hours, travel time, visa details	Source-check time-sensitive facts; display caveats
Image-rights mistakes	Wikimedia media licenses differ	Preserve attribution and file-page metadata
Dependency compromise	Package updates can change behavior	Versions/lockfiles, dependency scanning, review

16.2 Do not confuse Gradio password auth with full platform security

The `FLY_APP_PASSWORD` variable enables a simple application-level login in the current `launch(auth=...)` pattern. It helps avoid casual unrestricted use, but it is not a complete identity platform: it does not inherently provide per-user cost budgets, sophisticated rate limits, MFA, audit trails, or individualized data retention.

Protected/public Space visibility and app password protection are distinct features. Review the exposure of embed URLs and the visibility of code before sharing with classmates.

16.3 Handling sensitive user input

The current demo does not need passport numbers, credit-card data, access keys, medical records or exact home addresses to suggest attractions. The input form should avoid requesting such details. If a traveler enters sensitive information despite that, consider masking and deletion controls. This project is educational; no formal privacy compliance assessment is implied.

16.4 Source reliability hierarchy

For time-sensitive operational details, prefer **official attraction, transport, embassy/consulate or government sources** over model memory or general-purpose pages. Wikipedia is useful for descriptive context and photos but not a final authority for a restaurant reservation, visa requirement or current opening hours.

16.5 Financial safeguards

Set AWS Budgets and appropriate alarms, review model usage, limit agent steps and output size thoughtfully, and restrict access while testing. A single chat turn may generate **multiple Bedrock calls** during the CodeAgent loop; cost is not merely one model completion per user message.

16.6 Known technical limits of the inspected implementation

- No durable cross-session trip database or authentication-aware account profiles.
- Explicit corrections are in recent chat context, not always persisted into the canonical snapshot.
- Session history is truncated; very old preferences can drop out.
- A global execution lock serializes model requests and may reduce simultaneous-user capacity.
- Image matching uses heuristics; it can miss true places or include imperfect matches.

- Media attribution should be enhanced for production use.
- The model can still provide unsupported or inaccurate travel facts despite prompting.
- A CodeAgent executes model-generated actions; stronger sandboxing is advised for serious deployment.
- No guaranteed uptime or fixed response latency; depends on HF Space and AWS services.

---

17. Quality assurance and professor's evaluation

17.1 Recommended test architecture

Evaluate four distinct layers, rather than treating "the pink page loads" as a complete success:

Test layer	Example	What it proves
UI	Press Preview; inspect cards and buttons	Gradio can display outputs
Model/API	Test DeepSeek through Cell 6	AWS credentials, model ID and provider route work
Agent	Run a brief and inspect steps/ tools	Instructions, CodeAgent/tool workflow work
End-to-end	Generate → image links → Send → Reset	Combined application experience works

17.2 A/B test design

Keep the **same trip query and model** when comparing baseline and improved agents. If you change the model, date, tool availability, hardware or query wording, explain the confounding factor. Inference is partly stochastic, so consider repeating scenarios rather than assuming a single transcript measures all behavior.

Example scenario A — Initial planning

Prompt: “Help me plan a four-day trip to Kyoto, Japan. I enjoy temples and quiet places.”

Measure: number of questions asked before providing a meaningful plan; whether it provides a plan; relevance of place selection; explanations.

Example scenario B — Cautious inference

Turn 1: “I chose a countryside hotel last year because it was cheaper.”
Turn 2: “Where should I stay in Kyoto?”

Measure: whether the assistant infers a fixed countryside preference incorrectly; whether it labels the inference.

Example scenario C — Correction

Turn 1: “I prefer quiet accommodation.”
Turn 2: “Actually, I want a central location even if it is busy.”

Measure: whether the next answer honors the newer explicit instruction rather than reusing the obsolete preference.

Example scenario D — Targeted follow-up

After creating a day-by-day itinerary, ask: “How can I make Day 2 cheaper without changing the other days?”

Measure: whether the assistant gives a focused answer instead of regenerating the entire itinerary.

17.3 Quantitative metrics (with formulas)

These are proposed evaluation measures, not observed scores:

Question burden

[Q =]

Explanation coverage

[C_{explain} = %]

Correction success rate

[C_{correct}=%]

Useful-link coverage

[C_{links}=%]

First visible feedback latency

[L_{feedback}=t_{first visible status}-t_{submit}]

First usable itinerary latency

[L_{plan}=t_{first full itinerary}-t_{submit}]

Images can be measured separately so external Wikimedia latency is not confused with model generation speed.

17.4 Evidence table template

Test ID	Baseline observation	Improved observation	Metric	Supported conclusion
A1	Paste verbatim baseline evidence	Paste verbatim improved evidence	Number of questions	Record only after testing
A2	Original rationale snippets	Revised rationale snippets	Explanation coverage	Record only after testing
A3	Original correction response	Improved correction response	Correction success	Record only after testing
A4	Baseline time stamps	Improved time-stamps	First feedback and plan latency	Record only after testing

Academic integrity: Avoid turning the project's illustrative screenshots, sample preview text or expected output into fabricated empirical evidence. The write-up should cite actual saved transcripts and measured timings.

17.5 Functional acceptance checklist

- [ ] Code builds in the intended Gradio 6 version.
- [ ] Preview (no AWS) produces a labeled demonstration.

- [ ] AWS secrets are present without being printed.
- [ ] DeepSeek itinerary is returned for Kyoto and Tokyo examples.
- [ ] Custom duration such as `2 weeks` is accepted and normalized.
- [ ] Free-text Other interests influence the itinerary.
- [ ] Initial plan is readable, day-by-day and explained.
- [ ] At least one normal chat follow-up is answered in context.
- [ ] A correction does not trigger an unwanted full-plan rewrite.
- [ ] Missing photos do not hide the itinerary.
- [ ] Place links open relevant searches/pages.
- [ ] Reset clears the current state.
- [ ] A second browser session does not reveal the first user's plan.
- [ ] Logs and error status never expose credentials.

---

18. Troubleshooting — error messages, likely cause and resolution

18.1 Troubleshooting decision tree

```
Does the Space show a web page?
  NO  → Check README YAML, requirements and build/container logs.
  YES → Does Preview (no AWS) work?
          NO  → Check Gradio component compatibility and callback wiring.
```

```
        YES → Does Generate work?
                NO → Check AWS secrets, IAM, model region and backend logs.
                YES → Does Send work?
                        NO → Check session state, chat callback and agent run.
                        YES → Are photos missing?
                                YES → Check Wikimedia HTTP calls/title matching.
                                NO  → Run full acceptance tests.
```

18.2 Error reference

Error / symptom	Layer	Typical root cause	First action
`SignatureDoesNotMatch`	AWS authentication	Mismatched/stale key pair or signing setup	Rotate/recheck the matching key and region; do not paste secrets
`AccessDeniedException` on `bedrock:InvokeModel`	AWS IAM	Identity lacks permission or explicit deny exists	Inspect effective IAM policy and model ARN
`stopSequences` unsupported	Model request	Agent sends unsupported stop field	Configure `stop=REMOVE_PARAMETER`
`assignment_model` undefined	Notebook execution	Cell 6 not run in current runtime	Execute prerequisites in order
`UserInputTool` undefined	Notebook execution	Cell 8 not run or import failed	Reload definitions
`Chatbot(... type=...)` invalid	Gradio 6	Old argument removed	Remove `type="messages"`
`gradio.components.Component` missing	Gradio runtime	Inconsistent package/import state	Check versions; clean reinstall/restart if needed
`theme` / `css` warnings	Gradio 6	Settings passed to `Blocks()`	Move to `launch()`
Button click produces blank page	UI callbacks	Callback blocked on inference or yields no result	Add early progress yield and visible exception handler
Cell 16 never stops	Colab app server	Gradio keeps server running	Stop cell or use non-blocking launch pattern
Repository not found `404`	HF Hub	Space not created, wrong ID/type, or private access	Verify `whoami`, repo ID, Space type
Space creation `402`	HF hosting	Missing subscription/hosting eligibility	Use eligible hosting plan or source-only Static Space
README YAML validation `400`	HF metadata	`short_description` over limit	Shorten field and re-upload
“huggingface.co refused to connect”	Browser/iframe	Attempt to embed HF website where framing is blocked	Open Hub links in top-level/new browser tab
Static Space shows a page but no chatbot	Hosting SDK	Static HTML cannot run Python app	Deploy `sdk: gradio` Space
“Fly not connected to AWS”	HF env	Missing `AWS_ACCESS_KEY_ID` or `AWS_SECRET_ACCESS_KEY`	Configure Space Secrets and restart

Error / symptom	Layer	Typical root cause	First action
Model response slow	Bedrock/tool loop	Multiple inference steps and research	Inspect logs, steps, context size and AWS latency
No photographs in deployed chat	Media service	No image match, blocked HTTP, old `app.py`	Run image preview and inspect media logs
Blank `Chatbot` after error	UI/error handling	Exception before output update	Verify generator yields error text and status
Agent asks same questions again	Prompt/context	Mode confusion or missing snapshot	Inspect `snapshot`, Mode A/Mode B prompt

18.3 Troubleshoot one layer at a time

Example: Preview works, Generate fails. The Gradio page and basic callbacks are functioning. Focus on the backend: AWS environment variables, IAM permissions, region, model ID, LiteLLM request parameters and model error logs. Rebuilding the pink CSS would not solve this.

Example: Generate works, photos are missing. The agent and Bedrock route are functioning. Focus on `_wiki_media`, external HTTP access, place-title normalization and browser loading. Changing IAM permissions would not solve an unavailable Wikipedia photo.

Example: Model works in Colab, fails on Hugging Face. The hosted Space has a separate runtime, dependencies and environment variables. A valid Colab AWS session does not automatically configure the Space. Check the hosted `agent_backend.py`, pinned packages and Space Secrets.

18.4 How to write a useful bug report

Record: (1) which environment, (2) which button, (3) exact input without private details, (4) expected behavior, (5) actual error type/message, (6) most recent source commit and relevant library versions, and (7) whether Preview succeeded. Redact account identifiers, signed URLs, tokens, access keys and other private data before sharing logs.

---

19. Technical roadmap — beyond the current educational demo

19.1 Prioritized future improvements

Priority	Improvement	Business / UX value	Technical direction
P0	Explicit photo attribution and robust media fallbacks	Trust and legal clarity	Wikimedia file metadata, license/credit panel, caching
P0	Corrected preferences persisted in structured session data	More dependable R4	Explicit preference parser + user confirmation
P0	AWS usage limits	Control charges	Rate limiting, budgets, quotas, authenticated access
P0	Harden CodeAgent execution	Reduce attack surface	Sandboxing, restricted credentials, tool permissions
P1	Saved itineraries across sessions	Return to trip later	Auth + encrypted database + retention controls
P1	Structured itinerary output schema	More reliable UI cards	Validated JSON/Pydantic contract, strict parser
P1	Official travel-source verification	Better factual accuracy	Attraction/transport source connectors with citations
P1	Per-user concurrency and async jobs	Support more classmates	Proper request isolation / worker pool / queues
P2	Map itinerary explorer	Better geographical overview	Geocoded place IDs and mapping component
P2	Export to PDF / share link	Submission-ready plan	Report generator and access-controlled sharing
P2	Multilingual mode	More accessible	Language toggle and localized prompts
P2	Human preference feedback	Better recommendations	Clear dislike/save/correct UI controls

19.2 Why validated structured output is a major improvement

The current agent returns Markdown and an optional fenced block of attraction names. This is flexible but fragile: a model can omit or misspell metadata. A more mature design would request a validated itinerary schema with fields such as `destination`, `days`, `stops`, `name`, `reason`, `source_url`, `confidence`, and `verification_status`. The UI would render cards only from validated and independently verified values.

This would make the photos, maps, error states and assignment metrics more reproducible.

19.3 Productionization is not just visual polish

Professional UX includes graceful failures, truthful uncertainty, predictable corrections, accessible controls, privacy handling, security, monitoring, attribution, and user agency. A visually polished landing page is only one part of a trustworthy AI service.

---

20. Glossary and official references

20.1 Essential terminology

Term	Plain explanation
Agent	Model plus tools and an execution policy
API	Programmatic request/response interface
AWS Bedrock	Hosted API to invoke foundation models
IAM	AWS permission and identity management
STS	AWS temporary identity/session service
LiteLLM	Provider abstraction used by Smolagents model calls
`CodeAgent`	Smolagents agent that can execute generated Python actions
Tool	Named function/service the agent can invoke
Prompt	Instructions and context sent to the model
`final_answer`	Agent completion response
Gradio	Python library creating browser-based interfaces

Term	Plain explanation
`gr.State`	Session-specific data exchanged by Gradio callbacks
Callback	Python function invoked by a UI action
Generator	Python function yielding incremental results
YAML	Configuration format used at top of Space README
Space	Hugging Face hosted application repository/runtime
Secrets	Configuration values stored outside source code
Static Space	Space displaying static files; not a Python app server
Gradio Space	Space that installs and runs the Python Gradio app
Wikimedia	Content/media platform providing optional real imagery
Baseline	Original system preserved for comparison
A/B evidence	Comparable observed performance on same scenarios
R1–R4	Four human-centered redesign requirements
HAI	Human–AI Interaction

20.2 Official technical references

The following links are useful for verification and future maintenance. URLs are intentionally placed here in the document for durable reference; vendor interfaces and pricing can change.

1. AWS: DeepSeek V3.2 model card — IDs, regions, supported endpoints.
2. AWS: Bedrock model API compatibility — supported model API families.
3. Hugging Face: Spaces overview and visibility — Gradio, Static, private/protected/public, hosting.
4. Hugging Face: Space configuration reference — README YAML fields.
5. Hugging Face: Manage Spaces using `HfApi` — repository creation and control.
6. Hugging Face: Smolagents documentation — agent tools, models and CodeAgent.
7. Smolagents GitHub guide — model-parameter handling including `REMOVE_PARAMETER`.
8. Gradio: Gradio 6 migration guide — API changes to constructor and launch arguments.
9. Gradio: CSS and JS customization — UI customization and themes.

10. Microsoft Research: Guidelines for Human–AI Interaction — official HAI design principles.
11. Microsoft: Eighteen HAI design guidelines — explanation of guidelines.
12. CHI 2019 original HAI paper — academic reference.
13. Wikimedia API documentation — source media API guidance.
14. Hugging Face token settings — account token management.

20.3 Project files inspected to produce this handbook

- `HAII_Cell_14_Fly_Conversational_Agent.py` — notebook improved agent.
- `HAII_Cell_16_Fly_Pink_Conversational_Gradio6.py` — notebook pink chatbot.
- `Fly_Travel_Agent_HF_Pro_Gradio_Space/agent_backend.py` — hosted agent code.
- `Fly_Travel_Agent_HF_Pro_Gradio_Space/app.py` — original hosted app.
- `Fly_HF_Images_Enabled_app.py` — later proposed photo-enrichment patch.
- `Fly_Travel_Agent_HF_Pro_Gradio_Space/requirements.txt` — deployed dependency declarations.
- `Fly_Travel_Agent_HF_Pro_Gradio_Space/README.md` — original YAML plus setup instructions.
- `Fly_HF_PRO_Deployment_Colab.ipynb` — standalone upload notebook.
- The relevant notebook cells and deployment troubleshooting documented in the conversation.

---

21. Practical learning exercises

These exercises are designed so someone new to cloud development can demonstrate understanding instead of merely memorizing steps. Do them on an authorized development environment, not with uncontrolled real AWS billing.

Exercise 1 — Explain the end-to-end request

Explain, in your own words, what happens after the traveler presses **Design my journey**. Distinguish the roles of `app.py`, `agent_backend.py`, Smolagents, LiteLLM and Bedrock. A strong answer follows the data through the entire return path to the browser.

Exercise 2 — Identify two different authentication systems

Explain why a Hugging Face write token is required for publishing **but not automatically required for the model at runtime**, while AWS IAM credentials authorize the DeepSeek inference. Describe where each credential should be stored and why hardcoding it is unsafe.

Exercise 3 — Investigate one actual provider error

Imagine the Space opens and Preview succeeds, but Generate returns `AccessDeniedException`. Which system component should you inspect first? Which IAM actions and model ARN are relevant? Why would changing pink CSS be unrelated?

Exercise 4 — Model versus UI correction

A traveler first says they enjoy quiet gardens, then requests a busy central neighborhood. Identify which part of the prompt should prefer the newest correction, and why a future version should also update a structured state field if the preference must persist across many conversations.

Exercise 5 — Explain the missing-image issue

An itinerary appears but no photograph is visible. Explain why the cause may involve Wikimedia rather than Bedrock; identify the lookup function, the acceptable fallback, and the licensing considerations.

Exercise 6 — Compare baseline and improved interaction fairly

Run the same Kyoto prompt against your saved baseline and improved agent, preserving actual transcripts. Count initial questions, measure explanation coverage, and record whether a follow-up triggers an unnecessary full itinerary regeneration. Present the differences without inventing numerical results.

Exercise 7 — Design the next version

Sketch a product requirement for **“Save my trip and return next week”**. Specify persistent storage, user authentication, what data is saved, deletion/retention controls, and how the system would distinguish the newest preference update from an old inferred preference.

Final project summary

Fly’s central engineering lesson is that an AI travel application is **not just a prompt**. It is a multi-layer system combining **human-centered requirements**, structured user input, model orchestration, optional tools, a responsive web interface, isolated session state, external photo and map content, cloud credentials, hosted deployment and testing.

The student’s project progressed from a professor-provided baseline Smolagents notebook to a DeepSeek V3.2/AWS-backed conversational travel concierge, then to a standalone Gradio application on Hugging Face PRO. The important problems encountered—IAM access, unsupported request parameters, framework-version incompatibility, Static versus Gradio hosting, YAML validation, missing Space secrets and image retrieval—illustrate genuine software-engineering boundaries between the layers.

The most useful maintenance principle is simple: when something breaks, identify the layer first. Fix AWS authentication in AWS, model-request compatibility in the adapter, agent behavior in its instructions, UI interactivity in Gradio, publishing errors in Hugging Face metadata, and images in the Wikimedia-enrichment path. Preserve verified working components and test one change at a time.